HOTEL RESERVATION SYSTEM

System Architecture

Project	Hotel Reservation System
Document	System Architecture
Document ID	ARCH-001
Version	1.0
Status	Active
Owner	Engineering
Last Updated	2026-09-13

Official readable project artifact. Editable source is maintained under `docs/_source/`.

Contents

- Document control
- Purpose
- Architecture principles
- System topology
  - LOCKED
- Repository architecture
- Application boundaries
  - Frontend - `apps/web`
  - Backend - `apps/api`
  - Database - PostgreSQL
- Backend Modular Monolith
  - LOCKED organization principle
  - Module boundary rules
  - Explicitly rejected for V1
- REST API boundary
  - LOCKED
  - PROVISIONAL early API areas
- Core data direction
  - LOCKED concepts
  - PROVISIONAL
  - DEFERRED
- Reservation consistency and integrity
  - LOCKED reservation creation invariants
  - PROVISIONAL database enforcement direction
- Pricing model
  - LOCKED V1
  - DEFERRED
- Configuration and environment variables
  - LOCKED principles
  - PROVISIONAL configuration areas
- Security boundary
  - Public boundary
  - Admin boundary

    - Data privacy
- Environment strategy
- Deployment direction
    - LOCKED
    - PROVISIONAL
- Logging and observability
    - PROVISIONAL baseline
    - DEFERRED
- Media storage direction
    - LOCKED principle
    - DEFERRED
- Email integration direction
    - LOCKED principle
    - DEFERRED
- Decision register

Document control

Version	Date	Change	Author
1.0	2026-09-13	Initial architecture baseline for Sprint 1	Engineering

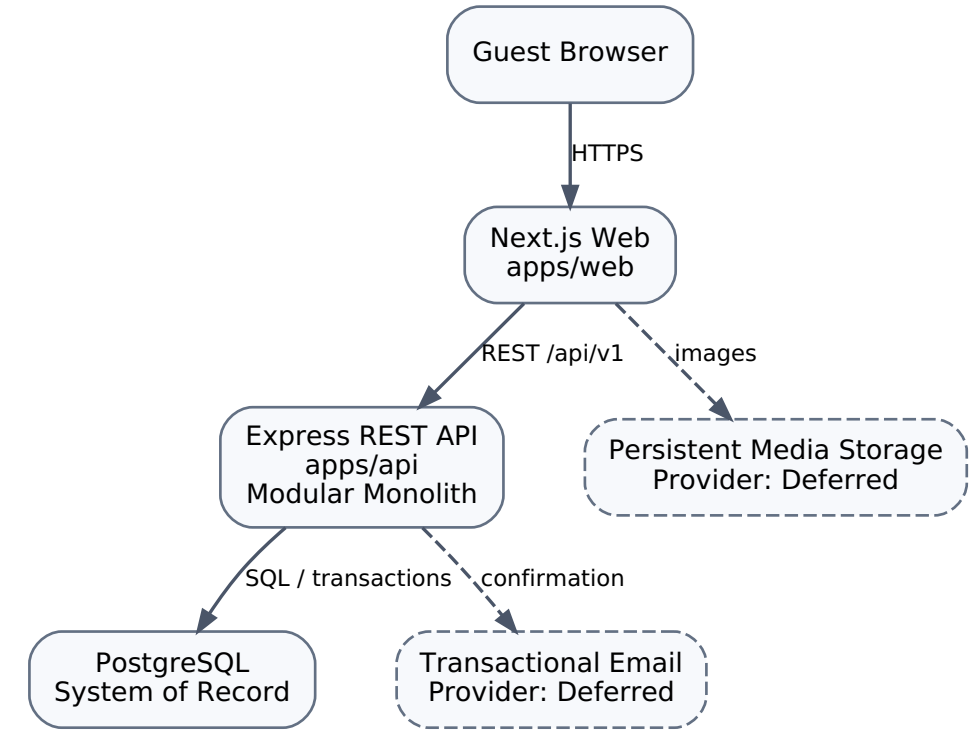
Purpose

This document records only the architecture needed to begin delivery safely. It intentionally avoids designing every future component. Decisions are labelled **LOCKED**, **PROVISIONAL**, or **DEFERRED** so the team can distinguish firm constraints from reversible choices.

Architecture principles

- Deliver vertical, testable product increments rather than frontend/backend/database phases.
- Keep the backend as a **feature-first Modular Monolith**, not microservices.
- Treat PostgreSQL as the final source of truth for reservation and inventory integrity.
- Keep public room-type booking separate from internal physical-room allocation.
- Prefer simple, reversible decisions until a requirement justifies more complexity.
- Security, testing, observability, deployment, and data integrity are delivery concerns, not later phases.

System topology



System topology

LOCKED

- **Frontend:** Next.js + TypeScript.
- **Backend:** Node.js + Express + TypeScript.
- **API:** REST.
- **Database:** PostgreSQL.
- **Deployment platform:** Railway.
- **Repository:** npm workspaces monorepo.
- **Application boundaries:** `apps/web` and `apps/api`.
- **Backend architecture:** feature-first Modular Monolith.
- **Public business API namespace:** `/api/v1`.
- **Service health endpoint:** `/health`.

Repository architecture

```
hotel-reservation/
├── apps/
│   ├── web/              # Next.js public + admin UI
│   └── api/              # Express modular monolith
├── docs/                 # official PDF docs + reproducible sources
├── .github/              # CI workflows when introduced
├── package.json          # workspace-level concerns only
└── workspace configuration
```

Runtime dependencies belong to the application that uses them. Shared monorepo packages are introduced only when actual cross-application reuse justifies them.

Application boundaries

Frontend - `apps/web`

Responsibilities

- Public hotel and room/package experiences.
- Availability search and reservation form UX.
- Booking confirmation UX.
- Admin dashboard UI when that capability is introduced.
- Responsive rendering and SEO-friendly public pages.
- Client-side validation for usability only.
- Calling the backend through configured API endpoints.

Non-responsibilities

The frontend is **not** authoritative for room allocation, availability, price, booking totals, booking reference generation, admin authorization, or reservation consistency.

Backend - `apps/api`

Responsibilities

- Business validation and REST API behavior.
- Room/inventory availability calculation.
- Conflict-safe reservation creation and physical room allocation.
- Server-side price calculation and historical snapshots.
- Admin authentication/authorization when introduced.
- Package/inventory rules when introduced.

- Transactional email orchestration when introduced.
- Safe persistence access and database transactions.
- Operational logging and API error handling.

Deployment boundary

The API is one application and one deployable backend service. Internal modules are **not** separately deployed services.

Database - PostgreSQL

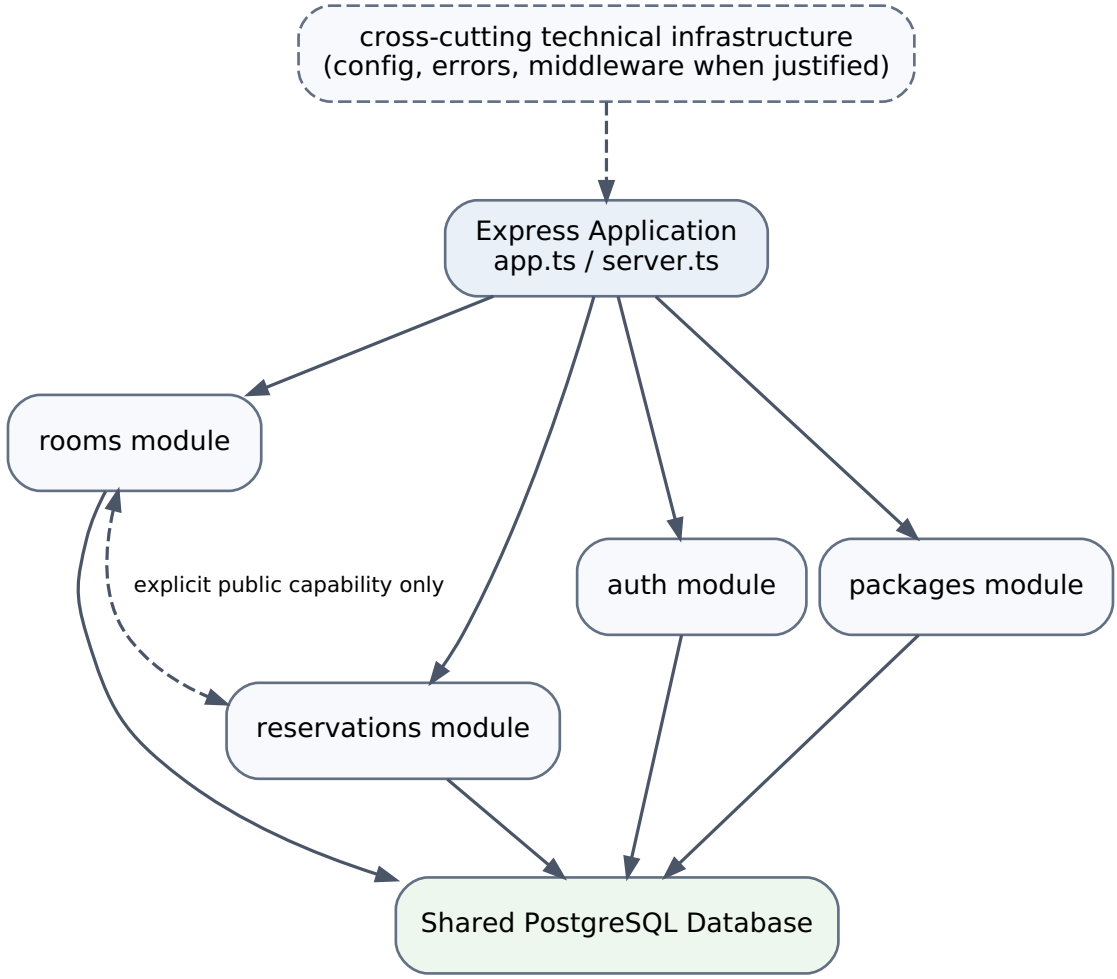
Responsibilities

- Durable system of record for room types, physical rooms, reservations, allocation, pricing snapshots, statuses, blocks, packages, admin users, and other delivered entities.
- Referential integrity and key business constraints.
- Final overlap/concurrency protection for reservation allocation.
- Transactional consistency.

Important constraint

The API may perform availability checks for user feedback, but database-backed reservation creation is the final authority. A stale availability response must never guarantee a booking.

Backend Modular Monolith



Modular Monolith

LOCKED organization principle

Business functionality is grouped by capability under a module-oriented structure such as:

```
apps/api/src/
├── modules/
│   ├── rooms/
│   ├── reservations/
│   ├── auth/
│   ├── packages/
│   └── ...
├── config/
├── shared/        # only when real cross-cutting code exists
├── database/      # introduced with persistence work
├── app.ts
└── server.ts
```

The structure is conceptual. Empty future modules and placeholder layers are not created in advance.

Module boundary rules

- A module owns business logic for its capability.
- Modules do not freely import one another’s internal files.
- If another module needs behavior, the owner exposes an intentional public capability.
- `shared/` is for genuinely cross-cutting technical concerns, not displaced business logic.
- One shared PostgreSQL database is used; database ownership should still respect capability boundaries where practical.
- A module does not require controller/service/repository/schema/type files unless the capability actually needs them.

Explicitly rejected for V1

- Microservices.
- Global layer-first business folders such as root-level `controllers/`, `services/`, `repositories/` containing mixed domains.
- CQRS/event-bus architecture.
- Generic base repositories.
- DI frameworks without a concrete need.
- Domain-event infrastructure without a concrete need.
- One npm package per business module.

REST API boundary

LOCKED

Business endpoints use a versioned namespace:

```
/api/v1/...
```

Operational service health remains separate:

```
/health
```

PROVISIONAL early API areas

Exact payloads and route names will be refined during implementation, but Sprint 1 will need capabilities equivalent to:

- room-type catalogue/details,
- availability search,
- reservation creation,
- booking confirmation response.

The public API must expose room-type/customer-safe information only; internal physical room numbers remain private during the guest booking flow.

Core data direction

LOCKED concepts

- `RoomType` - guest-facing category such as Deluxe Double or Family Room.
- `Room` - physical inventory unit owned by a room type.
- `Reservation` - one booking for one room.
- Reservation contact snapshot - guest/contact details stored for the booking; no guest account is required.
- Reservation status - created as Pending.
- Payment status - Unpaid / Pay at Hotel or Paid.
- Room allocation - internal physical room associated with the reservation.
- Price snapshot - accepted nightly price, nights, currency EGP, and total.
- Booking reference - unique guest-facing reservation reference.

PROVISIONAL

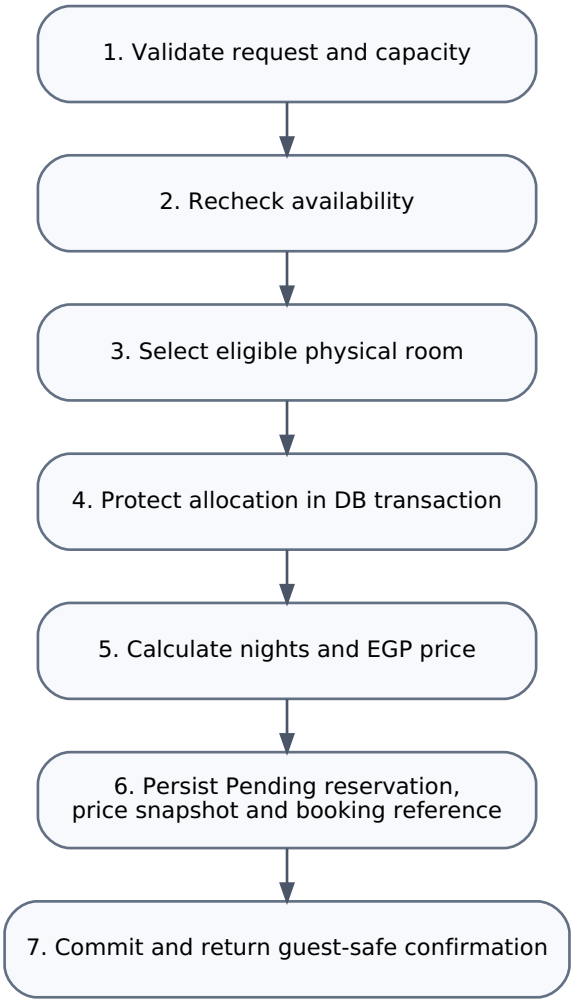
- `RoomBlock` will be introduced when availability needs staff/operational blocking behavior.
- A package reservation will reuse the same underlying room inventory/allocation rules instead of a separate availability engine.

DEFERRED

- Detailed seasonal/rate-plan model.
- Customer profile/CRM entity.
- Multi-room reservation model.
- Multi-currency model.

Reservation consistency and integrity

The stay interval is **[check-in, check-out)**. Checkout-day turnover is therefore valid.



Reservation integrity flow

LOCKED reservation creation invariants

1. Validate the request server-side.
2. Validate room-type capacity.
3. Recheck availability.
4. Find an eligible physical room.
5. Protect allocation inside a database transaction.
6. Prevent overlapping inventory-consuming reservations at the database level.
7. Calculate nights and pricing server-side.
8. Persist accepted price snapshot.
9. Create a Pending reservation and unique booking reference.
10. Commit.
11. Return guest-safe confirmation data only.

Two concurrent requests for the one remaining physical room over overlapping dates must result in **exactly one successful reservation**.

PROVISIONAL database enforcement direction

PostgreSQL range/constraint or transaction-locking facilities will be used to enforce the no-overlap invariant. The exact SQL/ORM approach is deliberately deferred until the persistence tool is selected, but the database-level guarantee itself is **LOCKED**.

Pricing model

LOCKED V1

- One current base nightly price per room type.
- Currency: EGP only.
- Reservation total = server-calculated nightly price x server-calculated nights.
- Reservation persists historical accepted pricing so future room-price changes do not alter existing bookings.

DEFERRED

Seasonal, weekend, holiday, promotional date-range, and advanced rate-plan logic.

Configuration and environment variables

LOCKED principles

- Configuration is environment-driven.
- Secrets are never committed to source control.
- Next.js client-exposed variables contain no secrets.
- The frontend API base URL is configurable.
- The backend uses server-only environment variables for database credentials, auth secrets, email credentials, and other sensitive configuration.
- Safe `.env.example`-style documentation may contain names and harmless examples only.

PROVISIONAL configuration areas

- Node runtime version will be pinned/documented when the repository is created.
- Exact environment validation library/tooling is not yet selected.

Security boundary

Public boundary

The public site and guest booking APIs are internet-facing. Guest requests are untrusted input. Public behavior must not accept authoritative client room allocation, price, total, or stale availability state.

Admin boundary

Admin routes and management APIs will require authenticated/authorized staff access. V1 uses one staff permission level. Secure credential hashing and secure session/token handling are required; sensitive auth material should not be stored in browser `localStorage`.

Data privacy

Guest PII is stored only as needed for the reservation and hotel operations. Normal logs must not unnecessarily record guest PII, secrets, password hashes, or credentials.

Environment strategy

Environment	Purpose	Direction
Local	Developer implementation and tests	npm workspaces; local env files ignored; local PostgreSQL once persistence starts
Staging	Integrated, inspectable Sprint increment	Railway; production-like configuration; used for Sprint Review and smoke validation
Production	Hotel live system	Railway + hotel domain; controlled secrets, backup/recovery awareness, HTTPS, logging/ monitoring

Deployment direction

LOCKED

Railway will host the Next.js application, Express API, and PostgreSQL. Application updates can be connected to the project Git repository. `DATABASE_URL` and other sensitive configuration are provided through secure environment variables.

PROVISIONAL

Exact Railway service topology, build commands, health configuration, and branch/environment promotion strategy will be finalized as the staging deployment task approaches.

Logging and observability

PROVISIONAL baseline

- Structured backend logs suitable for diagnosing failed requests and reservation issues.
- Health endpoint for service-level checks.
- Platform deployment/runtime visibility through Railway.
- Actionable error reporting without guest PII or secrets.
- Production smoke checks for the booking path.

DEFERRED

Dedicated APM/error-monitoring vendor, dashboards, tracing, and advanced metrics unless operational needs justify them.

Media storage direction

LOCKED principle

Production hotel/room/package/gallery media must use durable storage and must not rely on an ephemeral application filesystem.

DEFERRED

Provider, upload implementation, image transformations, CDN behavior, and detailed media-management workflow.

Email integration direction

LOCKED principle

Transactional reservation confirmation email is required in V1. Provider failure must not roll back an otherwise committed booking.

DEFERRED

Provider selection, retry mechanism, template tooling, and deliverability monitoring are chosen when PBI-006 approaches implementation.

Decision register

Decision	State	Rationale
Next.js frontend	LOCKED	Agreed project stack and SEO/ public-site fit.
Express REST API	LOCKED	Agreed project stack and explicit backend boundary.
PostgreSQL	LOCKED	Relational integrity and concurrency requirements fit the domain.
Railway hosting	LOCKED	Agreed deployment platform.
npm workspaces monorepo	LOCKED	Professional, conventional single-repo structure for two applications.
Feature-first Modular Monolith	LOCKED	Cohesive business modules without microservice complexity.
`[check-in, check-out)` dates	LOCKED	Correct hotel turnover semantics.
Database-level overlap protection	LOCKED	Prevents double booking under concurrency.
EGP / base nightly price	LOCKED	V1 business rule.
ORM/SQL access tool	DEFERRED	Select when persistence implementation begins.
Email provider	DEFERRED	Does not block Sprint 1 core booking transaction.
Media provider	DEFERRED	Does not block initial representative room fixtures.
Detailed admin UX	PROVISIONAL	Refine near PBI-005/PBI-008.
Advanced observability	DEFERRED	Start with minimal operational baseline.